Exception Handling

Day 4 Afternoon - Debugging, Exceptions, and Testing

30 min teaching + 30 min exercises

```
try:
    record_count = int(user_input)
except ValueError:
    print("Record count must be a whole
number.")
```

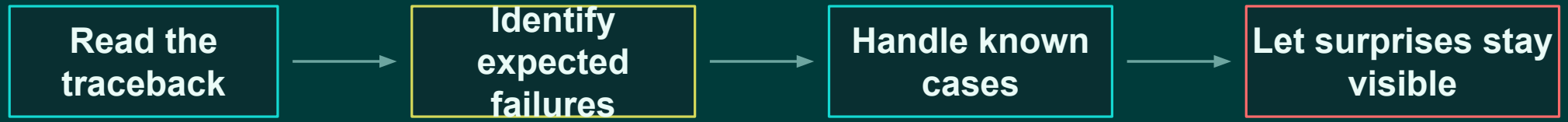
```
def calculate_percentage(completed,
total):
    if total <= 0:
        raise ValueError("total must be
greater than zero")

    return completed / total * 100
```

Goal: handle known failure conditions intentionally, without hiding real bugs.

BLOCK 30

Where this block fits

Block 29 taught how to diagnose failures. Block 30 teaches when and how to recover.



Instruction focus
- expected vs unexpected failures
- try and except
- specific exception types
- else and finally
- raising exceptions
- error recovery in record processing

Practice focus
Students convert unsafe operations into intentional, readable recovery paths.

BLOCK 30

Exception handling is not hiding errors

It is deciding which failures your program knows how to handle.

Bad goal
“Make errors disappear.”

Good goal
“Recover from known problems and expose unknown ones.”

Known problem

User typed
“twelve” instead of
12

Unknown bug

Programmer misspelled
variable name

Handle the first. Fix the second.

BLOCK 30

Expected failure: user input conversion

The code is valid, but the input may not be convertible.

```
record_count = int(input("Record count: "))
```

→ User enters twelve → Python raises ValueError

```
ValueError: invalid literal for int() with base 10: 'twelve'
```

Teaching point

Humans entering invalid input is not surprising. The program can respond with a readable message instead of crashing.

BLOCK 30

The basic try / except pattern

Put the risky operation in try, then handle the known exception.

```
try:
    record_count = int(
        input("Record count: ")
    )
except ValueError:
    print("Record count must be a whole
number.")
```

try block
The operation that might fail for an expected reason.

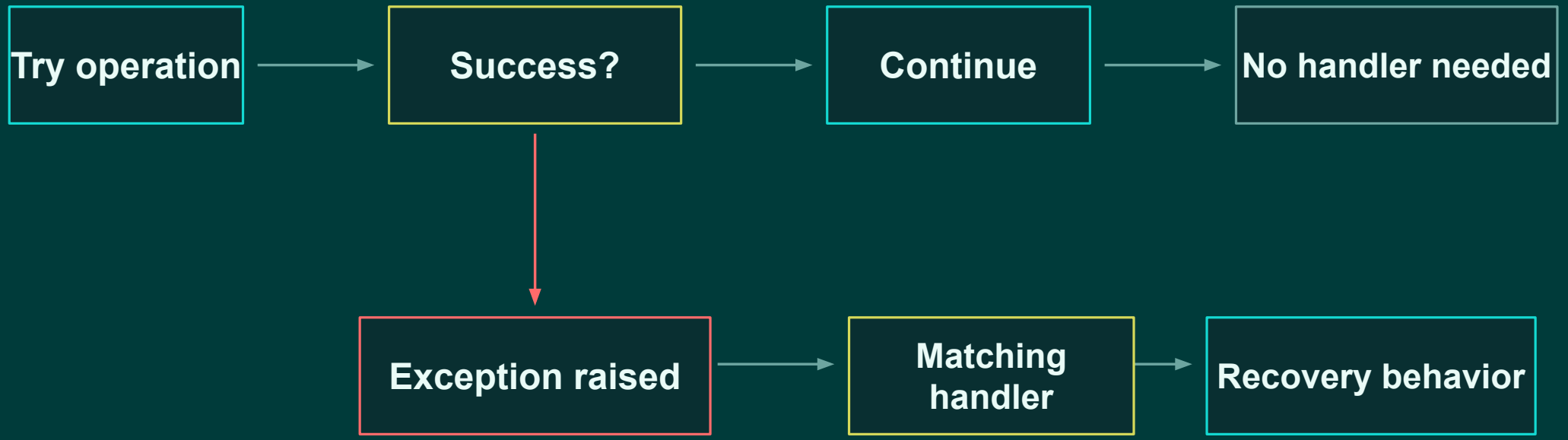
except block
The recovery path for one specific exception type.

Result
If conversion fails with ValueError, Python skips to the matching handler.

BLOCK 30

Mental model: controlled detour

An exception changes the path of execution.



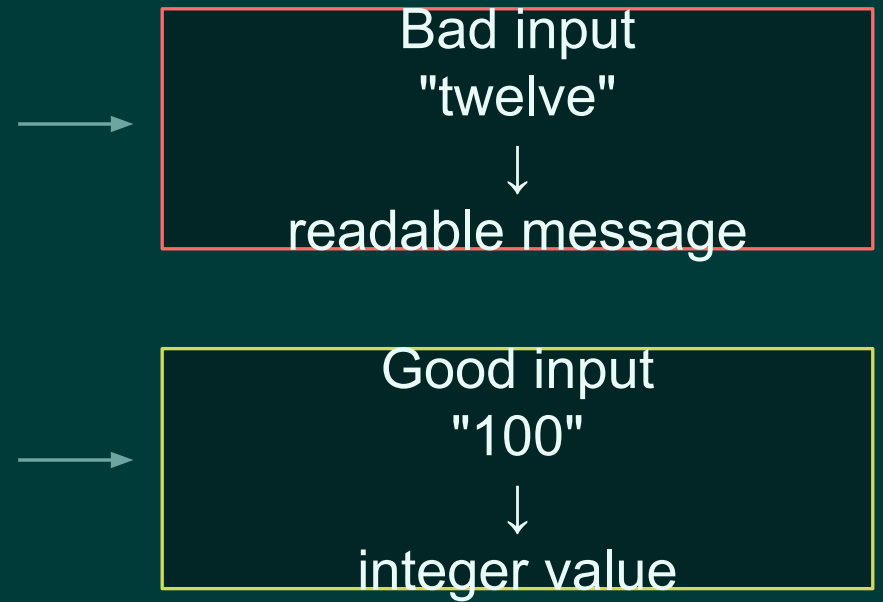
Big idea
A handled exception is not ignored. It is routed to code that knows what to do.

BLOCK 30

Safe conversion creates a clean boundary

Convert and validate before using a value in calculations.

```
user_input = input("Record count: ")

try:
    record_count = int(user_input)
except ValueError:
    print("Invalid record count")
else:
    print(f"Record count: {record_count}")
```



Keep unsafe text away from math until it becomes a real number.

BLOCK 30

Catch specific exceptions

Specific handlers preserve useful signals.

```
try:
    record_count = int(user_input)
except ValueError:
    print("Record count must be a whole
number.")
```

Preferred
Handles the failure you expect and understand.

```
try:
    record_count = int(user_input)
except:
    print("Something went wrong")
```

Risky
May hide misspelled variables, programming bugs, or unexpected system problems.

The goal is not fewer tracebacks. The goal is correct behavior.

BLOCK 30

Bare except can hide the real defect

A broad handler can turn a programmer mistake into a vague message.

```
try:
    percentage = completed / totla
except:
    print("Could not calculate percentage")
```

What really happened?
The variable name total was misspelled as totla.

Why this matters
The program did not recover from bad user data. It concealed a bug that should be fixed.

Beginner rule
Only catch exceptions you can explain and respond to intentionally.

BLOCK 30

Multiple exception types

Different failures may require different messages.

```
try:
    percentage = completed / total
except ZeroDivisionError:
    print("Total cannot be zero")
except TypeError:
    print("Values must be numeric")
```

ZeroDivisionError
The formula is valid, but total was zero.

TypeError
The operation received values that do not support division.

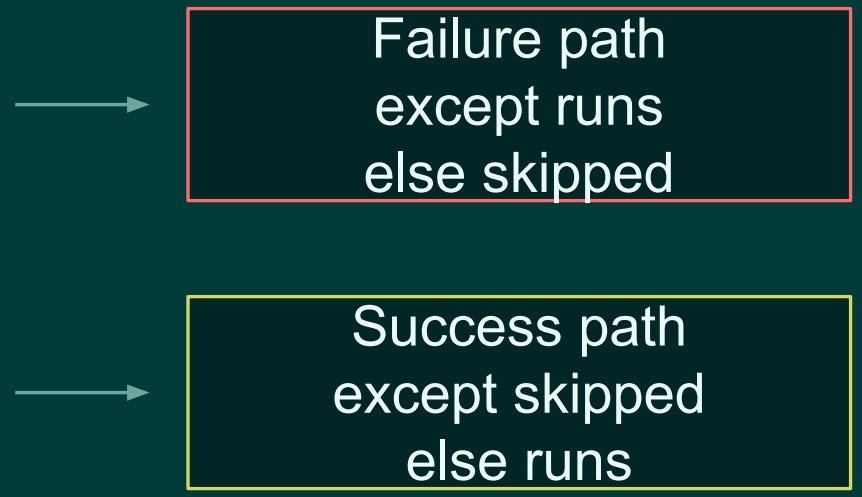
Why separate them?
Specific messages make the fix clearer for the user and the programmer.

BLOCK 30

The else block

else runs only when no exception occurs in the try block.

```
try:
    record_count = int(user_input)
except ValueError:
    print("Invalid record count")
else:
    print(f"Record count: {record_count}")
```



Use else for work that should happen only after the risky operation succeeds.

BLOCK 30

The finally block

finally runs whether the try block succeeds or fails.

```
try:
    print("Attempting operation")
except ValueError:
    print("Invalid value")
finally:
    print("Operation finished")
```

Success
Attempting operation
Operation finished

Handled failure
Invalid value
Operation finished

Teaching note
At this point, students only need the concept. File cleanup later makes finally more concrete.

BLOCK 30

Functions can reject invalid input

raise communicates that a function cannot meaningfully do its job.

```
def calculate_percentage(
    completed: int,
    total: int
) -> float:

    if total <= 0:
        raise ValueError(
            "total must be greater than zero"
        )

    return completed / total * 100
```

Why raise?
The function contract requires a meaningful total.

What ValueError says
The value was the wrong kind of value for this operation.

Important
Raising an exception is not crashing randomly. It is rejecting invalid input explicitly.

BLOCK 30

Function contracts and assumptions

Before raising errors, decide what the function promises.

Function
calculate_percentage(completed, total)

Inputs
completed: count finished
total: count possible

Assumption
total must be greater than zero

Output
completion percentage as a float

```
if total <= 0:
    raise ValueError(
        "total must be greater than zero"
    )
```

A clear contract makes exception behavior easier to explain and test.

BLOCK 30

Validation result or exception?

Not every undesirable value needs to crash the current record.

Business condition
Record contains 3 errors

↓

Return classification
"warning", "Minor errors detected"

Function violation
total is zero or negative

↓

Raise exception
ValueError("total must be greater than zero")

Use ordinary return values for expected business outcomes. Use exceptions for invalid assumptions.

BLOCK 30

Sometimes the right answer is not to catch it here

A function may raise an error and let the caller decide how to respond.

```
def calculate_percentage(completed, total):
    if total <= 0:
        raise ValueError("invalid total")

    return completed / total * 100
```



```
try:
    percentage =
calculate_percentage(18, 0)
except ValueError as error:
    print(f"Rejected record: {error}")
```

Why propagate?

The low-level function detects the invalid input. The higher-level code knows what message or record status to produce.

BLOCK 30

One bad record should not kill the whole batch

Exception handling lets processing continue when a record is malformed.

```
processed_records = []

for record in records:
    try:
        processed_record = process_record(record)
    except ValueError as error:
        print(f"Rejected record: {error}")
        continue

    processed_records.append(processed_record)
```

Normal record
process
append result

Bad record
report issue
continue loop

This is a real-world pattern: reject bad records without losing good records.

BLOCK 30

Instructor demo: build it in layers

Show exception handling as an intentional addition, not magic syntax.


```
record_count = int(input("Record count: "))
```

```
try:
    record_count = int(user_input)
except ValueError:
    print("Invalid record count")
```

Demo rule

Students should see the failure first, then see how the handler changes the behavior.

BLOCK 30

Guided exercises: 30-minute plan

Students practice handling expected failures without swallowing all errors.

0–6 min	6–12 min	12–17 min	17–23 min	23–30 min
Exercise 1 Safe integer conversion	Exercise 2 Safe division	Exercise 3 Dictionary access	Exercise 4 Raise exception	Exercise 5 Catch + message

Instructor goal

Keep students focused on evidence: what exception happened, why, and what response is appropriate?

Every handler should have a reason.

BLOCK 30

Exercise 1: safe integer conversion

Convert user text into an integer without crashing on bad input.

```
user_input = input("Record count: ")

# Add try / except here
record_count = int(user_input)

print(record_count)
```

Test values
100
-5
12.5
hello

Task
Catch ValueError and print a readable message. Do not use bare except.

6 minutes

Checkpoint
Which inputs convert successfully? Which produce ValueError?

BLOCK 30

Exercise 2: safe division

Decide how zero should be handled before writing the code.

```
def calculate_average(total, count):
    return total / count

print(calculate_average(100, 4))
print(calculate_average(100, 0))
```

Question
Should count = 0 return None, print a message, or raise ValueError?

Task
Implement the behavior your group chooses. Be ready to explain why.

6 minutes

Exception design starts with deciding what correct behavior means.

BLOCK 30

Exercise 3: dictionary access

Compare expected missing data with unexpected structure problems.

```
record = {
    "record_id": "EQ-1001"
}

print(record["status"])
print(record.get("status"))
```

Task

Run both access patterns. Record the difference between KeyError and None.

Discuss

When should a missing key reject a record? When should it indicate a programming bug?

5 minutes

BLOCK 30

Exercise 4: raise an exception

Make invalid totals explicit inside the function.

```
def calculate_percentage(completed, total):

    # Reject invalid totals here

    return completed / total * 100
```

Task
If total <= 0, raise ValueError with a helpful message.

Test calls
calculate_percentage(10, 20)
calculate_percentage(10, 0)
calculate_percentage(10, -5)

6 minutes

Checkpoint
The function should not quietly return nonsense for impossible inputs.

BLOCK 30

Exercise 5: catch the raised error

Call the function from code that can decide what message to show.

```
try:
    percentage = calculate_percentage(10, 0)
except ValueError as error:
    print(f"Rejected record: {error}")
else:
    print(f"{percentage:.1f}%")
```

Task
Test one successful call and one rejected call.

Explain
Which code detected the invalid input? Which code decided what to print?

7 minutes

Detection and recovery do not always belong in the same function.

BLOCK 30

Applied challenge: resilient record processing

Improve the processor so malformed input does not stop the entire dataset.

Challenge task
Process every record. If one record raises ValueError, print a rejection message and continue with the next record.

Required pattern
try
except ValueError as error
continue
append only successful records

```
for record in records:
    try:
        processed_record =
process_record(record)
    except ValueError as error:
        print(f"Rejected record: {error}")
        continue


processed_records.append(processed_record)
```

Remaining time

BLOCK 30

Applied challenge starter data

Use good records and malformed records so both paths are exercised.

```
records = [
    {"record_id": "EQ-1001", "completed": 18, "total":
24},
    {"record_id": "EQ-1002", "completed": 5, "total":
0},
    {"record_id": "EQ-1003", "completed": 42, "total":
50},
]

def process_record(record):
    percentage = calculate_percentage(
        record["completed"],
        record["total"]
    )

    return {
        "record_id": record["record_id"],
        "percentage": percentage,
    }
```

Task
Add the try / except loop around process_record(). One bad total should not stop the others.

Expected result
EQ-1001 and EQ-1003 are processed. EQ-1002 is rejected with a readable message.

Challenge

BLOCK 30

Block 30 wrap-up

Exception handling makes failure behavior intentional.

Students can explain
- expected vs unexpected failures
- try and except
- specific exception types
- else and finally
- raising ValueError

Students can build
- safe input conversion
- guarded calculations
- readable error messages
- record loops that continue after rejection

Big idea

Handle errors you expect and understand. Do not hide errors you need to fix.

Next: assertions and tests give us a systematic way to prove behavior.